Prompt Engineering: Prompt Ingredients and Prompting Strategies

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain what a prompt is and why the way you ask shapes the answer you get
- Decide when a task welcomes a model's creativity and when it does not
- Build a complete prompt from the five ingredients: role, task, context, constraints and output format
- Apply five basic strategies to sharpen any prompt you write

2. Detailed Explanation

What a prompt is, and what prompting means

A prompt is the input or instructions you give an AI model in natural language. It doesn't matter which model you use. It could be Gemini, an OpenAI model such as ChatGPT, or any other.

The flow is simple. Your input goes to the AI model, and the model sends back output.

```
User writes a prompt—> AI model reads the instructionsModel—> returns output
```

Say you type "help me learn Python". The model receives that task and produces a reply. Asking for a PPT, or for content for an upcoming presentation, works the same way. Anything you want the AI to do goes through a prompt.

Prompting is the process of creating a prompt. You can also think of it as a skill: giving instructions so clearly that the LLM understands your specific requirement in detail.

Prompting is really just communication, and human communication follows the same rules. Read this jumbled sentence: "Ate Aditi an apple." It's hard to follow because it breaks grammar rules. Now read "Aditi ate an apple." Same words, instantly clear. A prompt with clear structure gets understood in the same way.

Why prompt quality matters

Here's something that surprises many beginners. The same model does not always give the same quality of answer. The way you ask decides the kind of reply you receive.

This is the principle of garbage in, garbage out, which means a poor question rarely brings back a good answer.

Good prompts pay off in several ways:

- They produce the best possible, precise output.
- They avoid unnecessary information.
- They save time by getting correct information quickly.
- They give the model a direction of thought, so output arrives the way you want it.
- They provide context and say exactly what you want. Without that, output can be incomplete.
- They specify role, context, task, format and constraints, so the output matches reliably. That list is itself a prompting framework, and you'll meet it in detail below.

Let's see the difference with two prompts about SQL.

	Prompt	What you get
Weak	"Tell me something about SQL."	A generic overview
Strong	"Explain SQL to a first-year engineering student who has never learned it before. Use simple language with as little jargon as possible. Give three examples."	A focused, tailored explanation

The second prompt wins for four clear reasons:

1. It states the actual task: explain SQL.
2. It names the target audience, a first-year engineering student. That's the context.
3. It sets rules to follow, such as simple language and little jargon. Those are constraints.
4. It asks for examples, which shapes the output.

Jargon means using too many specialist keywords and terms. A word like "temperature" inside a prompt is one example of such a technical term.

Guessing versus not guessing

Is a model that guesses more better, or one that guesses less? It's a fair question.

Unnecessary guessing has real costs. Every prompt uses electricity and water, for example to cool server centres, and that adds to the carbon footprint. Guessing can also lead to hallucinations.

Still, "guessing is always bad" is wrong, and "guessing is always good" is wrong too. The honest answer is it depends on the task at hand. Prompt engineering serves many tasks beyond coding and content creation. It helps to read "guessing" here as creativity or imagination.

Creativity is welcome (subjective, creative tasks)	**Creativity is not welcome (logic-driven, rule-based tasks)**
Storytelling and creative writing	Maths, such as $(a + b)^2 = a^2 + 2ab + b^2$
Advertisements, which people appreciate when creative	Code, where a hello world program should simply print "hello world"
Blog writing and marketing	Facts, such as which city is the capital of India
Image generation, art, music and graphic design	Medical reasoning
	Legal matters and legal documents
	Science and physics laws

A quick way to sort a task: ask how much room for subjectivity it has. Little room means you want the model to stay precise.

The five ingredients of a good prompt

Before we look at the ingredients, clear up one myth. Longer prompts are not automatically better. You want a relevant prompt, not a large one. Limited but relevant information with clear instructions makes a better prompt.

A good prompt has five ingredients: role, task, context, constraints and output format.

Role

The role tells the model which persona or character to take on.

Think about everyday life. You visit a doctor for medical issues, an engineer for problem solving, an interior designer for design, and a dentist for your teeth. Each specific need goes to a specific profession. A role in a prompt works the same way.

Some example roles:

- Python instructor for beginners
- SQL developer
- Software development manager
- Financial advisor
- Medical professional

Task

The task tells the model exactly what it should do. The more specific the task, the better the reply.

- "Teach me SQL" is too broad. "Teach me the concept of joins in SQL" is better.
- "Teach me Python" is too broad. "Teach me the object-oriented programming concept in Python" is better.

Context

Context is background information. It describes the situation in which the task should happen. The same prompt produces different outcomes in different contexts.

Take one fixed task, "Give me a five-slide presentation on ML", and change only the context:

- For non-technical first-year MBA students: the model produces basic slides with little jargon. It covers ideas like ML being about identifying patterns, and rule-based versus non-rule-based approaches.
- For a final-year interview presentation with an IT firm: the model introduces the names of different machine learning models.

Other quick contrasts include a workout plan for three months versus six months. Another is learning Python for an interview versus for college exams.

Watch out for: confusing a task with context. "Create an image of a black cat drinking milk" is a task. It says what to do, not the situation around it.

Constraints

Constraints set limits for the model to work within. They reduce guesswork and act like a guardrail.

- "Give me a straightforward answer in three lines."
- "Explain Gen AI within 200 words for an MBA student" versus "Write a blog of at least 500 words for tech enthusiasts."

That second pair mixes constraint and context, but it shows nicely how limits change the result.

Output format

The output format states the shape of the response. You could ask for "Explain Gen AI" as:

- One single paragraph
- A summary
- A table
- Exactly 10 bullet points

Format is different from constraints. Format answers questions like "Should the roadmap be a table, a paragraph or a summary?"

Putting it together: a complete prompt for learning Python lists

Now let's build a real prompt. The goal is to learn lists in Python from an AI, and each ingredient is chosen on purpose.

Ingredient	Choice	Why this choice
Role	"Act as a beginner-friendly Python instructor."	You want to learn, not solve a project problem, so an instructor fits better than a developer. "Beginner-friendly" matches the context.
Task	"Explain lists in Python."	It targets the specific list concept, not Python in general. Small details matter.

Context	First-year engineering students who have never programmed or coded before	It tells the model who is reading.
Constraints	Simple language, no jargon or advanced concepts, three practical examples	It keeps the answer within clear limits.
Output format	A variety of code examples	"Include code" can act as either a constraint or a format.

Here is the assembled prompt:

```
Act as a beginner-friendly Python instructor. Explain lists in Python
to first-year engineering students who have never programmed before.
Use simple language, avoid jargon and advanced concepts, give three
practical examples, and include a variety of code examples.
```

Notice that it reads as one natural query. There's no single correct prompt, and yours might be worded differently.

Watch out for: labelling each part separately with tags like "Role:" or "Constraint:". That's a common mistake. Each element should simply fit where it belongs in the sentence.

Five basic prompting strategies

These five strategies are simple but powerful, and they build directly on the ingredients you just learned.

Strategy 1: be as specific as possible. "Explain Gen AI" is vague. A stronger version says who it's for, how much detail to give, and asks for examples: "Explain Gen AI to a student who has never studied it, in detail, with examples."

Strategy 2: specify the target audience. Anyone who has worked in sales and marketing knows the target audience is central. Without a stated audience, the model can't tailor the answer well. Naming the audience is part of the context.

Strategy 3: give relevant examples. This works best for content creation and anything personalised. Code is objective. About 90% of people could write the same piece of code, and a hello world program looks almost identical for everyone. A LinkedIn post or an email is personal, though, because vocabulary, word choice and writing style differ between people.

So give the model three or four samples of your own writing. It then learns the vocabulary, words and tone to use. Skip this, and an AI-generated email is easy to spot because it doesn't match your usual tone. A simple instruction does the job: "Use the below email as a reference."

Strategy 4: specify the tone. Tone is especially useful for content generation.

- Professional, for an email to a manager
- Friendly, for a birthday invite
- Educational, for YouTube course content such as a thumbnail
- Simple and natural vocabulary

Strategy 5: specify the length. Try "Write a blog within 100 words" or "Explain AI upscaling in five bullet points."

Tone and length both belong to the output format ingredient. Ever noticed that many LinkedIn creators' posts follow a fixed pattern, like always having five bullet points? That happens because they generate content with AI while specifying their tone and format.

3. Key Takeaways

- A prompt is the natural-language input you give an AI model, and prompting is the skill of creating it clearly.
- Prompt quality decides output quality: garbage in, garbage out. A specific SQL prompt gets a far better answer than a vague one.
- Whether a model's guessing helps depends on the task. Creative work welcomes it, while maths, code, facts, medical, legal and science tasks do not.
- Relevance beats length. A good prompt combines role, task, context, constraints and output format into one natural query.
- Five strategies sharpen prompts further: be specific, name the audience, give examples, set the tone and set the length.

Mental model: Think of prompting like choosing the right professional and briefing them well. You pick the doctor, engineer or designer who fits the need, then tell them the job, the situation, the limits and how you want the result delivered.